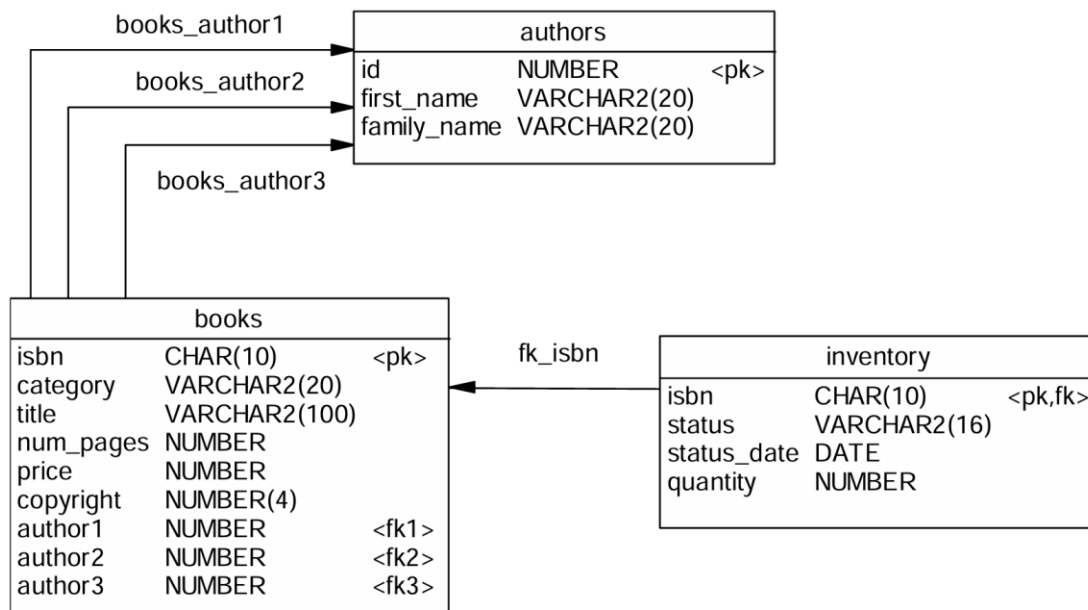


The schema used for this session:



User-defined Data Type:

For the above schema, we need to define a function that takes an author's first name as input and returns the author's full name and the number of books he has authored. Since a function can return only one value, we can bypass this limitation by creating a user-defined record type that contains two fields: `full_name` and `book_count`.

```
SET SERVEROUTPUT ON;

--Creating that return type

CREATE OR REPLACE TYPE author_info_type AS OBJECT
(
    full_name VARCHAR2(50),
    book_count NUMBER
);

/

--Creating the function

CREATE OR REPLACE FUNCTION get_author_info(fname IN authors.first_name%TYPE)
RETURN author_info_type IS
    fam_name VARCHAR(25);
    fullname VARCHAR(50);
    b_count NUMBER;
```

```
BEGIN
    SELECT family_name INTO fam_name FROM authors WHERE first_name = fname;

    SELECT COUNT(*) INTO b_count FROM books
    WHERE author1 = (SELECT id FROM authors WHERE first_name = fname)
       OR author2 = (SELECT id FROM authors WHERE first_name = fname)
       OR author3 = (SELECT id FROM authors WHERE first_name = fname);

    fullname := fname || ' ' || fam_name;
    RETURN author_info_type(fullname, b_count);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN author_info_type('No author found', 0);
END;
/

--Calling the function

DECLARE
    result author_info_type;
BEGIN
    result := get_author_info('Robert');
    DBMS_OUTPUT.PUT_LINE(CHR(10) || 'Full Name: ' ||
result.full_name);
    DBMS_OUTPUT.PUT_LINE('Books Authored: ' ||
result.book_count);
END;
/
```

Arrays:

We will find and store the ISBNs of books authored by a given author in an array and print them.

Using fixed size array:

```
DECLARE
    TYPE isbn_arr_type IS VARRAY(5) OF books.isbn%TYPE;
    isbn isbn_arr_type :=
        isbn_arr_type(NULL, NULL, NULL, NULL, NULL);
```

```
author_id authors.id%TYPE;
idx INTEGER := 1;
BEGIN
    SELECT id INTO author_id FROM authors WHERE first_name =
    'Robert';

    FOR rec IN (
        SELECT isbn FROM books
        WHERE author1 = author_id
            OR author2 = author_id
            OR author3 = author_id
        )
    LOOP
        isbns(idx) := rec.isbn;
        idx := idx + 1;
    END LOOP;

    FOR i IN 1..idx-1
    LOOP
        DBMS_OUTPUT.PUT_LINE('ISBN: ' || isbns(i));
    END LOOP;
END;
/
```

Using dynamic array/nested table:

```
DECLARE
    TYPE isbn_tab_type IS TABLE OF books.isbn%TYPE;
    isbns isbn_tab_type := isbn_tab_type();
    author_id authors.id%TYPE;
    idx INTEGER := 1;
BEGIN
    SELECT id INTO author_id FROM authors WHERE first_name =
    'Robert';

    FOR rec IN (
        SELECT isbn FROM books
```

```
        WHERE author1 = author_id
              OR author2 = author_id
              OR author3 = author_id
    )
    LOOP
        isbns.EXTEND;
        isbns(idx) := rec.isbn;
        idx := idx + 1;
    END LOOP;

    FOR i IN 1..isbns.COUNT
        LOOP
            DBMS_OUTPUT.PUT_LINE('ISBN: ' || isbns(i));
        END LOOP;
    END;
/
```

Trigger:

A trigger is a PL/SQL block that executes automatically when a specific event occurs.

PL/SQL triggers have two main forms:

1. Application triggers – Execute on events in Oracle Forms (like button clicks or form navigation).
2. Database triggers – Execute on database events, such as table modifications (INSERT, UPDATE, DELETE) or schema events (logon, DDL).

There are two main levels of triggers:

1. Row-level: These triggers execute once for each row affected by the triggering DML statement (INSERT, UPDATE, or DELETE).
2. Statement-level: These triggers execute only once per DML statement, regardless of the number of rows affected by that statement.

In Oracle PL/SQL row-level triggers, :OLD and :NEW are special pseudorecords that give access to the values of a row before and after a DML operation.

1. :OLD
 - a. Refers to the existing values in the row before an UPDATE or DELETE.
 - b. Available only in UPDATE and DELETE triggers.

2. :NEW

- a. Refers to the new values in the row being inserted or updated.
- b. Available in INSERT and UPDATE triggers.

Let's consider our books table. It has a column 'price'. We want to ensure that the price is always within a valid range before inserting or updating.

```
CREATE OR REPLACE TRIGGER check_book_price BEFORE INSERT OR
UPDATE ON books
FOR EACH ROW
DECLARE
    p_min CONSTANT NUMBER(8,2) := 10.0;
    p_max CONSTANT NUMBER(8,2) := 100.0;
BEGIN
    IF :NEW.price < p_min OR :NEW.price > p_max THEN
        RAISE_APPLICATION_ERROR(-20001, 'Book price is out of
allowed range');
    END IF;
END;
/
```

Look at this trigger and try to understand what it does:

```
CREATE OR REPLACE TRIGGER log_price_change AFTER UPDATE OF price
ON books
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Price changed for ISBN ' || :OLD.isbn
|| ' from ' || :OLD.price || ' to ' || :NEW.price);
END;
/
```

We want to categorize the book price (Low, Medium, High). Every row of books table will have a category associated with it. For that, we have to add a column 'price_category' to the books table, and then make a trigger that will automatically generate the category and set that category value for the 'price_category' column.

```
ALTER TABLE books ADD price_category VARCHAR(8);

--Creating the trigger

CREATE OR REPLACE TRIGGER tr_price_category BEFORE INSERT OR
UPDATE ON books
FOR EACH ROW
BEGIN
    IF :NEW.price > 60 THEN
        :NEW.price_category := 'High';
    ELSIF :NEW.price > 40 AND :NEW.price <= 60 THEN
        :NEW.price_category := 'Medium';
    ELSE
        :NEW.price_category := 'Low';
    END IF;
END;
/

--Now run these statements and see the changes

SELECT isbn, price, price_category FROM books;

update books set copyright = 0;

select isbn, price, price_category from books;
```

THE END